Ders 2 Genişletilmiş Notu: Doğrusal Yoklamalı Hash Tablosunda Ortalama Yerleşme Adımının Hesaplanması

Ders 2 Genişletilmiş Notu: Doğrusal Yoklamalı Hash Tablosunda Ortalama Yerleşme Adımının Hesaplanması

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Sorunun Hatırlatılması
- 2. İlk Öneriler ve Neden Tutmadıkları
- 3. Tablodan Hesap: Değer ile Göz Numarasını Karşılaştırmak
- 4. Birinci Sadeleştirme: Eşitlik Kontrolünü Kaldırmak
- 5. İkinci Sadeleştirme: if Yapısını Kaldırmak
- 6. Genel Algoritma
- 7. Hashing Neden Kullanılır?
- 8. Sınavda Sık Görülen Verimsiz Çözüm
- 9. Doğruluk Yetmez: Verimlilik ve Sınav Stresi
- 10. Son Kontrol ve Üç Aşamalı Özet
- 11. Akış Diyagramında Döngü Okları

1. Sorunun Hatırlatılması

Bu kayıt haftanın uygulama dersidir. Hoca, pazartesi günkü derste sorduğu düşünme sorusunu öğrencilerle birlikte çözdü. Soru, üçüncü sınıfta Algoritma Analizi dersinde anlatılan hashing konusundan geliyor. Hoca hashing'in kendisini burada anlatmadığını, şu anda önemli olanın sorudaki algoritma olduğunu özellikle belirtti. Yöntemin amacı, elemanlara çok az arama yaparak, neredeyse doğrudan erişebilmektir.

1.1 Yerleşme Kuralı

Uzunluğu $N = 10$ olan bir tablomuz (table) ve yerleştirilecek 5 elemanımız olsun: 23, 4, 3, 9, 29. Hashing'de kullanılan yöntemlerden biri şöyle çalışır:

- Gelen x elemanının N'e göre modu alınır ve x mod N numaralı göze bakılır.
- Göz boşsa eleman oraya yerleştirilir.
- Göz doluysa ardışık gözlere bakılır ve bulunan ilk boş göze yerleştirilir.
- Tablonun sonuna gelinirse başa, yani 0. göze dönülür.

  Ek bilgi (derste anlatılmadı): Dolu gözde ardışık gözlere bakarak boş yer arama yöntemine doğrusal yoklama (linear probing) denir. İki elemanın aynı göze düşmesine çarpışma (collision) adı verilir. x mod N gözüne de elemanın ev adresi (home address) denir. Bu notta "ev adresi" terimi bu anlamda kullanılır.

Elemanlar sırayla şöyle yerleşir:

- **23:** $23 \bmod 10 = 3$. 3. göz boş, oraya yerleşir.
- **4:** $4 \bmod 10 = 4$. 4. göz boş, oraya yerleşir.

- **3:** $3 \bmod 10 = 3$. 3. göz dolu, 4. göz dolu, 5. göz boş. 5. göze yerleşir.
- **9:** $9 \bmod 10 = 9$. 9. göz boş, oraya yerleşir.
- **29:** $29 \bmod 10 = 9$. 9. göz dolu. Tablonun sonundayız, başa döneriz. 0. göz boş, oraya yerleşir.

Sonuçta oluşan tablo ve her elemanın kaçıncı denemede yerleştiği şöyledir:

Göz	Eleman	Ev adresi	Kaçıncı adımda yerleşti
0	29	9	2 (9 dolu, 0 boş)
3	23	3	1
4	4	4	1
5	3	3	3 (3 dolu, 4 dolu, 5 boş)
9	9	9	1

Diğer gözler (1, 2, 6, 7, 8) boştur.

1.2 Sorunun Kendisi

Soru şudur: Elemanlar ortalama kaç adımda yerleşmiştir? Tablo sırasıyla adımları toplayalım: 2, sonra $2+1=3$, sonra $3+1=4$, sonra $4+3=7$, sonra $7+1=8$. Toplam 8 adım, 5 eleman vardır:

$$\text{ortalama} = \frac{2+1+1+3+1}{5} = \frac{8}{5} = 1{,}6$$

Hoca bu değeri "yaklaşık 1,6 adım" diye söyledi. $8/5$ tam olarak $1{,}6$'dır.

Sorunun can alıcı noktası, elimizde **yalnızca yerleştirilmiş tablonun** olmasıdır. Elemanlar yerleştirilirken adım sayılmamıştır. Hesap bu hazır tablodan yapılmalıdır.

2. İlk Öneriler ve Neden Tutmadıkları

2.1 Yerleştirirken Saymak

Bir öğrenci, elemanların bulunduğu bir A dizisi ve ondan iki kat uzun bir tablo alarak işe başlamayı önerdi. Yerleştirme sırasında adımları saymıştı.

Soru: Elemanları tabloya yerleştirirken adımları sayarak ortalamayı bulamaz mıyız?

Cevap: Hayır. A dizisi artık yoktur, sayılar çoktan yerleşmiştir. Elde yalnızca tablo vardır. Soru, bu tablodan her elemanın kaçıncı adımda yerleştiğinin nasıl bulunacağıdır.

2.2 İstatistiksel Tahmin

Başka bir öğrenci soruya istatistik yönünden yaklaştı. Toplam adım sayısının en az 5, en fazla $n(n-1)/2$ olabileceğini söyledi. Düzgün dağılımda beklenen değerin iki sınırın ortalaması olduğundan yola çıkarak $(10+5)/2 = 7{,}5$ adım sonucuna ulaştı.

Hoca bunun tutmadığını gösterdi: Bu tabloda gerçek ortalama $8/5$'tir, 2 bile değildir. Üstelik 5 elemanlı bu durumda bir eleman en fazla 5 adımda yerleşebilir. Burada istenen matematiksel bir açıklama ya da tahmin değil, **gerçekten hesaplamaktır**.

> **Ders sonrası not:** Öğrenci üst sınırı önce 15, sonra işlemde 10 olarak kullandı. $n=5$ için $n(n-1)/2 = 10$'dur. Bütün elemanlar aynı ev adresine düşerse adımlar $1, 2, 3, 4, 5$ olur ve toplam $n(n+1)/2 = 15$ çıkar. Yani doğru üst sınır 15'tir. Ayrıca bu sınırlar elemanın değil, toplamın sınırlarıdır. Bu yüzden 7,5 ile gerçek toplam olan 8 karşılaştırılmalıdır; hocanın söylediği "en fazla 5 adım" ise tek bir elemanın sınırıdır. Yine de böyle bir tahmin, tablodaki gerçek değeri vermez.

Hoca bu arada hashing'in özelliğine de değindi: Tablo uzunluğu eleman sayısının iki katı alınırsa, elemanlar eleman sayısıyla büyümeyen küçük bir adım sayısında yerleşir. Bu konuya 7. bölümde dönülecek.

3. Tablodan Hesap: Değer ile Göz Numarasını Karşılaştırmak

Bir öğrenci doğru fikri önerdi: Elemanın modunu alıp olması gereken gözü bulalım, bulunduğu gözle arasındaki uzaklığa bakalım. Hoca fikri tahtaya şöyle döktü. i. gözdeki eleman için

$$value = table[i] \bmod N$$

hesaplanır. `value`, elemanın olması gereken gözdür. Elemanın gerçekte bulunduğu göz ise i'dir. Üç durum vardır.

3.1 Eleman Yerinde: value = i

23 için $i = 3$, $value = 23 \bmod 10 = 3$. Değer ile göz numarası eşittir. Eleman ilk adımda yerleşmiştir: `step = 1`. Aynı şey 4. gözdeki 4 ve 9. gözdeki 9 için de geçerlidir.

3.2 Eleman Aşağı Kaymış: value < i

5. gözdeki 3 için $i = 5$, $value = 3$. Eleman 3. gözde olmalıydı, 5. göze kadar kaymış. Bu durumda

$$step = i - value + 1 = 5 - 3 + 1 = 3$$

olur. Gerçekten de 3, 4 ve 5. gözlere bakılmıştır. Sondaki $+1$'in nedenini hoca şöyle açıkladı: İlk yerleşme denemesi sıfırıncı değil birinci adımdır. İlk yerleşme için de bir efor harcanmıştır.

3.3 Uç Nokta: value > i

Öğrenci bu noktada takıldı: 0. gözdeki 29 "işi bozuyor". $i = 0$, $value = 9$'dur; $i - value$ negatif çıkar. Kısa bir tartışmanın ardından tablo uzunluğunu eklemek gerektiği görüldü:

$$step = N + i - value + 1 = 10 + 0 - 9 + 1 = 2$$

Gerçekten de 29, 9. ve 0. gözlere bakarak 2. adımda yerleşmiştir.

> **Hocanın vurgusu:** Bir problemi çözerken sayılarla deneyin. En güzel ve hızlı çözüm yolu budur, problemi çoğu zaman gözle görebilirsiniz. Uç noktalara da dikkat edin. Buradaki uç nokta 29'dur: 3'ün 5. göze yerleşmesi şaşırtıcı değildir, daha büyük bir adrese kaymıştır. 29 ise ezberi bozup tablonun başına dönmüştür. Bu durum dikkate alınmazsa çözüm yanlış olur.

Bu ilk çözüm, hocaya göre temel olarak doğru ve temiz kafayla önerilmiş oldukça başarılı bir çözümdür:

```
value = table[i] % N
eğer value == i ise:
    step = 1
değilse, eğer value < i ise:
    step = i - value + 1
değilse:
    step = N + i - value + 1
```

4. Birinci Sadeleştirme: Eşitlik Kontrolünü Kaldırmak

Hoca sonra "Bunu kısaltabilir misiniz?" diye sordu. Bir öğrenci cevabı buldu.

Soru: `value == i` kontrolüne gerçekten gerek var mı?

Cevap: Yok. $value = i$ olduğunda $i - value + 1$ ifadesi zaten 1 verir. 23 için $3 - 3 + 1 = 1$. Bu yüzden ayrı bir kontrol yapmak yerine koşul "küçük eşit" yapılır ve iki durum tek dalda toplanır.

Böylece çözüm iki dala iner:

```
value = table[i] % N
eğer value <= i ise:
    step = i - value + 1
değilse:
    step = N + i - value + 1
```

Hoca bu hâli üç durumla da sınadı:

- $i = 3$**:** `table[3] = 23`, $value = 3$. $3 \leq 3$ olduğu için $step = 3 - 3 + 1 = 1$. Doğru.
- $i = 5$**:** `table[5] = 3`, $value = 3$. $3 \leq 5$ olduğu için $step = 5 - 3 + 1 = 3$. Doğru.
- $i = 0$**:** `table[0] = 29`, $value = 9$. $9 > 0$ olduğu için $step = 10 + 0 - 9 + 1 = 2$. Doğru.

> **Ders sonrası not:** Hoca tahtada tablo uzunluğunu bir ara M, bir ara N yazdı ve derste birkaç kez düzeltti. Doğrusu her yerde tablo uzunluğu olan N'dir.

5. İkinci Sadeleştirme: if Yapısını Kaldırmak

Hoca sıradaki soruyu sordu: Bu `if`'ten de kurtulabilir miyiz? Bir öğrenci önerdi: İki dalda da N ekleyelim, sonra sonucun N'e göre modunu alalım.

N'i neden eklediğimizi hatırlamak gerekir. N, tablonun dairesel (circular) yapısından gelir: 9. göz doluysa başa dönülür, yerleştirme 0. gözden devam eder.

Soru: Tabloya şimdi 39 gelse kaçıncı göze yerleşir?

Cevap: 1. göze. Çünkü $39 \bmod 10 = 9$; 9. göz dolu, 0. göz dolu, 1. göz boştur.

Ancak N'i her duruma eklersek, $value \leq i$ olan elemanlar için sonuç bozulur. $i = 5$ için $10 + 5 - 3 + 1 = 13$ çıkar ve biz 13 değil 3 istiyoruz. Öğrencinin önerdiği mod işlemi bu fazlalığı atar:

$$step = (N + i - value + 1) \bmod N$$

Hoca formülü iki durumla sınadı:

- $i = 1$**, eleman 39:** $value = 39 \bmod 10 = 9$. $step = (10{+}1{-}9{+}1) \bmod 10 = 3 \bmod 10 = 3$. Gerçekten de 39, 9. gözde 1., 0. gözde 2., 1. gözde 3. denemede yerleşmiştir.
- $i = 5$**, eleman 3:** $value = 3$. $step = (10 + 5 - 3 + 1) \bmod 10 = 13 \bmod 10 = 3$.

Aynı formül tablodaki bütün elemanlar için şu sonucu verir (ara adım eklendi):

Göz i	Eleman	$value$	$N + i - value + 1$	$step$
0	29	9	2	2
3	23	3	11	1
4	4	4	11	1
5	3	3	13	3
9	9	9	11	1

Toplam yine 8, ortalama yine $8/5 = 1{,}6$'dır. Mod sayesinde tek bir ifade hem $value \leq i$ hem de $value > i$ durumunu çözer.

> **Ders sonrası not:** Bu tek satırlık formülün bir uç durumu vardır. Eleman tam N. denemede yerleşmişse, yani i ev adresinin dairesel olarak hemen bir öncesindeki gözse, $N + i - value + 1$ değeri N ya da $2N$ olur ve mod işlemi N yerine 0 verir. Bu yalnız tablo tamamen dolduğunda, son yerleşen eleman için görülebilir. Rastgele tablolarla yapılan denemede formül yalnız bu durumda şaştı; iki dallı `if` çözümü ise hiç şaşmadı. Her durumda doğru sonuç veren yazım $step = (N{+}i{-}value) \bmod N{+}1$ ifadesidir. Dersteki örnek tablolarda bu durum oluşmaz.

6. Genel Algoritma

Hoca formülü bütün tablo için genelleştirdi. N elemanlı bir tablo verilmiştir. Dikkat edilecek iki nokta vardır:

- Her gözde eleman olmayabilir. Önce gözün dolu olup olmadığına bakılır. Boş gözde iş yapılmaz. Hesaptaki `if`'ler kaldırılabilir ama bu kontrol kaldırılamaz.
- Yalnız toplam tutmak yetmez. Ortalama alabilmek için kaç dolu göz olduğu da sayılmalıdır. Hoca bunu sayacı yazmayı unutup hatırlayarak gösterdi: "Sıfırdan farklı kaç eleman olduğunu bilmezseniz ortalama da alamazsınız."

```
N, table[N] oku
sum = 0
count = 0
i = 0'dan N-1'e kadar:
    eğer table[i] != NULL ise:
        value = table[i] % N
        step  = (N + i - value + 1) % N
        count++
        sum += step
average = sum / count
```

Hoca ilk yazışında `step` satırını `value` hesaplanmadan yazmıştı. Bir öğrenci "Buradan önce `value` hesaplayacaktık sanırım" diyerek uyardı, hoca da satırı ekledi. Kuruluş sırası önemlidir: `step`, `value`'ya bağlıdır.

> **Ek bilgi (derste anlatılmadı):** Derste C kodu yazılmadı. Aşağıdaki program, akış diyagramının ve 8. bölümdeki verimsiz çözümün C'ye çevrilmiş hâlidir; `gcc -Wall` ile uyarısız derlenmiş ve çalıştırılmıştır. `NULL` bir işaretçi değeri olduğu için tamsayı tablosunda boş göz ayrı bir işaretle (burada -1) gösterilmiştir. Çıktı `5 eleman: 1.60 1.60` ve `6 eleman: 1.83 1.83` satırlarıdır; ikinci tablo 39'un eklenmiş hâlidir ($11/6$).

```c
#include <stdio.h>

#define N 10
#define BOS (-1)   /* boş göz işareti; elemanlar negatif değil */

/* Derste varılan çözüm: her elemanın adımı tek bir ifadeyle bulunur. */
double ortalama_adim(const int table[], int n) {
    int i, value, step, sum = 0, count = 0;
    for (i = 0; i < n; i++) {
        if (table[i] != BOS) {                /* yalnız dolu gözler */
            value = table[i] % n;             /* ev adresi */
            step = (n + i - value + 1) % n;   /* kaçıncı denemede yerleşti */
            count++;
            sum += step;
        }
    }
    return (double)sum / count;
}

/* Derste eleştirilen çözüm: yeri bilinen eleman ev adresinden yeniden aranır. */
double ortalama_adim_verimsiz(const int table[], int n) {
    int i, value, step, adres, sayac, sum = 0, count = 0;
    for (i = 0; i < n; i++) {
        if (table[i] != BOS) {
            value = table[i] % n;
            if (value == i) {
                step = 1;
            } else {
                sayac = 0;
                adres = value;
                while (sayac < n && table[adres] != table[i]) {
```

```
                adres = (adres + 1) % n;
                sayac++;
            }
            step = sayac + 1;
        }
        count++;
        sum += step;
    }
}
return (double)sum / count;
}

int main(void) {
    int t1[N] = {29, BOS, BOS, 23, 4, 3, BOS, BOS, BOS, 9};
    int t2[N] = {29, 39, BOS, 23, 4, 3, BOS, BOS, BOS, 9};
    printf("5 eleman: %.2f %.2f\n", ortalama_adim(t1, N),
           ortalama_adim_verimsiz(t1, N));
    printf("6 eleman: %.2f %.2f\n", ortalama_adim(t2, N),
           ortalama_adim_verimsiz(t2, N));
    return 0;
}
```

7. Hashing Neden Kullanılır?

Verimsiz çözüme geçmeden önce hoca, hashing'de neyin amaçlandığını hatırlattı. Çünkü bir sonraki çözümün neden bu kadar rahatsız edici olduğu buradan anlaşılır.

Bilgisayar bilimlerinde en çok sıkıntı yaşanan işlemlerden biri aramadır ve arama olabildiğince hızlı yapılmak istenir. İkili aramada (binary search) bile karmaşıklık $O(\log n)$'dir, yani eleman sayısına bağlıdır. Hashing ise n'e bağlı olmadan, diyelim 3 ya da 5 adımda aranan elemana ulaşmayı hedefler. Yaklaşımın güzelliği budur.

Hoca bunu sayıyla da söyledi: Tablo boyutu eleman sayısının en fazla iki katı yapılırsa, örneğin 10 eleman için 20 uzunluklu tablo kullanılırsa, aranan eleman en fazla üç veya dört adımda bulunur. Bunun istatistiksel bir karşılığı vardır: doğum günü paradoksu (birthday paradox), yani aynı anda iki kişinin doğum gününün aynı olma olasılığı mantığından yola çıkan bir hesap. Ayrıntısı üçüncü sınıfta Algoritma Analizi dersinde anlatılır.

> **Ders sonrası not:** Hocanın "maksimum üç veya dört adım" ifadesi bir üst sınır olarak değil, tipik davranış olarak okunmalıdır. Bütün elemanlar aynı ev adresine düşerse, n. eleman n. denemede yerleşir; yani en kötü durum tablo boyutundan bağımsız olarak n adımdır. Hocanın 2. bölümdeki "5 elemanda en fazla 5 adım" sözü de bunu gösterir. Küçük ve n'den bağımsız olan, ortalama adım sayısıdır.
>
> **Ek bilgi (derste anlatılmadı):** Doğrusal yoklama için Knuth'un çözümlemesine göre, hash fonksiyonu elemanları düzgün dağıtıyorsa ve tablonun doluluk oranı α ise, beklenen yoklama sayısı başarılı aramada yaklaşık $\frac{1}{2}\left(1+\frac{1}{1-\alpha}\right)$, başarısız aramada yaklaşık $\frac{1}{2}\left(1+\frac{1}{(1-\alpha)^2}\right)$'dir. Tablo boyutu eleman sayısının iki katı ise $\alpha = 1/2$ olur ve bu değerler $1{,}5$ ile $2{,}5$ çıkar. Python benzetimi de bu değerleri verdi. Doğum günü paradoksu ise, tablo boş görünse bile çarpışmaların neden kaçınılmaz olduğunu açıklar.

8. Sınavda Sık Görülen Verimsiz Çözüm

Bu soru üçüncü sınıf öğrencilerine, Algoritma Analizi sınavında sorulmuştu. Hoca, üçüncü sınıfların veri yapılarını ve programlama derslerini geçmiş, algoritma geliştirmede bir adım daha tecrübeli olduklarını da hatırlattı. Şimdi konuşulan türden bir çözüm onlar için ilk akla gelebilecek bir şeydi. Oysa öğrencilerin yaklaşık yüzde 80'i başka türlü çözmüştü. Hocaya göre doğru çalışan ama "baya kötü" olan bu genel çözümü göstermek istemesinin sebebi, belli bir noktadan sonra algoritmanın doğruluğunun ötesinde verimli (efektif) olmasının da önemli olmasıdır.

8.1 Çözümün Yazılışı

```
value = table[i] % N
eğer value == i ise:
    step = 1
değilse:
    count = 0
    adres = value
    (count < N) VE (table[adres] != table[i]) olduğu sürece:
        adres = (adres + 1) % N
        count++
    step = count + 1        (ara adım eklendi)
```

Buradaki `count`, 6. bölümdeki dolu göz sayacından farklıdır; arama adımlarını sayar. Çözümün mantığı şudur: Eleman yerinde değilse, ev adresinden başla ve elemanı tabloda ara. Kaç gözden geçtiğini say.

> **Ders sonrası not:** Kısa notta ekrandaki koşulun `table[adr] != value` olduğu kayıtlıdır. Hoca ise anlatırken 00:37:11'de "table adres farklıdır table i" dedi ve böyle izledi. Doğrusu `table[adres] != table[i]` olmalıdır. Çünkü `value` ile karşılaştırılırsa 39 için arama 9. gözdeki 9 değerinde hemen durur ve 3 yerine 1 adım bulunur; 29 için de 2 yerine 1 çıkar.

8.2 Adım Adım: 39 ve 13

Hoca önce 39 elemanını izledi. 39'un 1. gözde olduğu tabloda:

- $i = 1$, $value = 39 \bmod 10 = 9$. $9 = 1$ mi? Hayır.
- `adres = 9`. 9. gözden başlanarak 39 aranır.
- 9. gözde 39 mu var? Hayır. Bir kere say.
- Adres artar, 0. göze gelinir. 39 mu var? Hayır. Bir kere daha say.
- 1. gözde 39 mu var? Evet. Arama biter.

Sayaç 2 olur, yani eleman 3. adımda yerleşmiştir (ara adım eklendi). Sonuç doğrudur. Oysa $i = 1$'deyken 39'un burada olduğu zaten biliniyordu. Adım sayısını bulmak için önce 9. göze gidilmiş ve az önce yeri bilinen 39 bir kez daha aranmıştır.

Sonra hoca aynı işi, 6. göze yerleşmiş varsayılan 13 için tekrarladı:

- $i = 6$, $value = 13 \bmod 10 = 3$. $6 = 3$ mü? Hayır.
- Sayaç sıfırlanır, `adres = 3`.
- 3. göz: 13 mü? Hayır. 4. göz: 13 mü? Hayır. 5. göz: 13 mü? Hayır. 6. göz: 13 mü? Evet.

Dördüncü adımda bulunur. Formül de aynı sonucu verir: $(10 + 6 - 3 + 1) \bmod 10 = 4$ (ara adım eklendi). Hoca bu çözüm için "40 yıl düşünsem böyle bir çözüm aklıma gelmezdi" dedi.

Soru: Her seferinde tablonun `adres mod N`. gözüne bakılıyor, değil mi?

Cevap: Evet, haklısın. Hoca yazarken burada da M ile N'i karıştırmıştı; doğrusu `adres = (adres + 1) % N` ifadesidir.

8.3 Daha Kötüsü: Gereksiz "Eleman Yoksa" Kontrolü

Hocaya göre çözümde birden fazla ciddi sorun vardır:

- **Yeri bilinen eleman yeniden aranıyor.** Herkes 39'un nerede olduğunu biliyor. Yine de 9'dan yola çıkılarak aranıyor.
- **`count < N` kontrolü gereksiz.** Bu kontrol "39 tabloda yoksa" diye konmuş.

Bu ikinci sorunu hoca pazartesi günkü derste konuşulan doğrusal aramayla (linear search) bağladı. Orada `i < n` ve `a[i] != aranan` koşulları vardı. `i < n` kontrolü, aranan eleman dizide yoksa döngünün gereksiz yere dönmesini engeller, n bitince dışarı çıkılmasını sağlar. O kontrol orada anlamlıdır. Burada ise aynı kontrol, yanlış yere taşınmıştır. Hocanın benzetmesiyle, 39 sanki hareket edebilen, gezebilen bir elemanmış gibi "ya 39 yoksa" kontrolü konmuştur. Oysa 39 bir adım önce bulunmuştur. Aramanın bir yerde onu bulacağı bilinir, yani koşul her zaman doğru çıkacak bir sonuca bakmaktadır.

Ders sonunda hoca sohbet ekranındaki yorumları okudu. Yorumlara göre bu çözümü yazanlar dinlediklerini uygulamaya çalışmış, ama özümseyememiş ve sayılara güvenememişlerdi. Çözüm için "primitif ama çalışıyor" yorumu da yazılmıştı. Hoca, çalıştığı tespitine katıldı.

> **Ek bilgi (derste anlatılmadı):** İki çözümün iş yükü farkı sayılabilir. Dersteki çözüm her göz için sabit iş yapar, toplam $O(N)$'dir. Verimsiz çözüm ise her kaymış eleman için ayrıca `step` kadar göz okur. Dersteki 5 elemanlı tabloda bu fazladan $2 + 3 = 5$ okumadır. En kötü durumda, bütün elemanlar aynı ev adresine düşmüşse, fazladan okuma sayısı $\frac{n(n+1)}{2} - 1$ olur ve iş yükü $O(n^2)$'ye çıkar. Sayımlar python ile doğrulanmıştır.

9. Doğruluk Yetmez: Verimlilik ve Sınav Stresi

> **Hocanın vurgusu:** Doğru çalışan bir algoritma kadar, yazdığınız algoritmanın verimli olması da önemlidir. Bu çözümü bugün özellikle paylaşmamın sebebi budur.

Hoca bunu bir dengeyle tamamladı. "Sınavda çok verimli, karmaşıklığı çok az bir algoritma yazmam gerekir" stresine hiçbir zaman kapılmamak gerekir. Sınavda her zaman iyi çözümler akla gelmeyebilir, bu çok doğaldır. Ama çözüm belli bir eşik seviyesinin altında kötü olmamalıdır. Böyle bir çözüm, bir iki dakika düşünüp "ben ne yapıyorum?" deyip kendinize gelmenizi gerektirir.

Hiç olmazsa şu ikisi yapılabilir: Eleman tabloda olduğuna göre `count < N` kontrolü konmaz, çünkü `table[i]`'de bulunan elemanı bir daha bulamama ihtimali yoktur. Ve yeri bilinen eleman yeniden aranmaz. Kişi aşama aşama kendini düzeltmelidir.

10. Son Kontrol ve Üç Aşamalı Özet

Hoca, M ve N'leri bir yerlerde karıştırdığını söyleyerek son algoritmayı iki sayıyla tekrar denedi.

- **5. göze yerleşmiş varsayılan 23:** $value = 23 \bmod 10 = 3$. $step = (10 + 5 - 3 + 1) \bmod 10 = 13 \bmod 10 = 3$. Doğru.
- **1. gözdeki 39:** $value = 39 \bmod 10 = 9$. $step = (10 + 1 - 9 + 1) \bmod 10 = 3$. 9. göz 1. adım, 0. göz 2. adım, 1. göz 3. adımdır. Doğru.

Ardından çözümün üç aşamasını özetledi:

Aşama	Yapı	Hocanın değerlendirmesi
1	`value == i` ayrıca kontrol edilir, kalan iki durum için dallanılır	Doğru, gayet iyi bir çözüm
2	`value <= i` ve `value > i` diye iki dal	Doğru, verimli, güzel bir çözüm
3	$step = (N + i - value + 1) \bmod N$, tek ifade	Biraz daha iyisi, görülmese de büyük sorun değil

Üçüncü aşamada `if` tamamen kalkar. Kalan tek `if`, gözün dolu olup olmadığına bakan ve kaldırılamayan kontroldür. Çünkü gözde eleman yoksa iş yapılmamalıdır.

11. Akış Diyagramında Döngü Okları

Ders sonunda bir öğrenci, akış diyagramında döngülerin nasıl çizildiğini sordu.

Soru: Döngünün içini sağdaki gibi, `N-1`'e kadar sayıldıktan sonraki çıkışını da herhangi bir şart belirtmeden soldaki gibi mi çizeceğiz?

Cevap: Evet. `for` kutusunda her zaman aynı düzen vardır:

- Kutuya yukarıdan girilir.

- Gövde bittikten sonra kutuya geri dönülür.
- Döngü bitince kutunun çıkış okundan dışarı çıkılır.

`while` için de karar kutusu kullanılır, burada şart açıkça yazılıdır:

- Yukarıdan koşula gelinir.
- Şart doğruysa (true) gövdedeki işler yapılır ve koşula geri dönülür. Örneğin şart `i < n` olabilir.
- Şart yanlışsa (false), yani bozulduğunda, döngüden dışarı çıkılıp aşağı gidilir.

Dersin Toparlaması

Bu uygulama dersinde, doğrusal yoklamalı bir hash tablosunda elemanların ortalama kaçıncı denemede yerleştiği yalnızca hazır tablodan hesaplandı. Temel fikir, her elemanın değerinden olması gereken gözü bulup bulunduğu gözle karşılaştırmaktır. Çözüm üç durumlu `if` yapısından iki dala, oradan da $step = (N + i - value + 1) \bmod N$ tek ifadesine indirildi. Boş gözler atlanır, dolu gözler sayılır ve ortalama `sum / count` ile bulunur. Dersteki tablo için sonuç $8/5 = 1{,}6$'dır.

Karşı örnek olarak, yeri bilinen elemanı ev adresinden yeniden arayan ve gereksiz "eleman yoksa" kontrolü koyan çözüm incelendi. Dersin ana mesajı şudur: Sayılarla deneyin, uç noktalara bakın ve doğru çalışan çözümle yetinmeyip onu sadeleştirin. Hashing'in istatistiksel temeli ise üçüncü sınıftaki Algoritma Analizi dersinde işlenecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.